Programação Concorrente

Threads

Professor: **Jeremias Moreira Gomes**
E-mail: jeremias.gomes@idp.edu.br

Threads

Relembrando o Modelo de Processo

- Softwares são organizados como processos sequenciais
- Cada processo possui sua CPU virtual
- Cada processo possui um espaço de endereçamentos e uma única linha (*thread*) de execução

Modelo de *Thread*

- Um espaço de endereço com a **possibilidade de múltiplas linhas de controle**
- Conjuntos de *threads* compõem as linhas de execução de um processo
- Threads **compartilham** um **mesmo espaço de endereço**
- Possuem recursos particulares
  - PC, registradores, pilha, etc

Threads - Vantagens

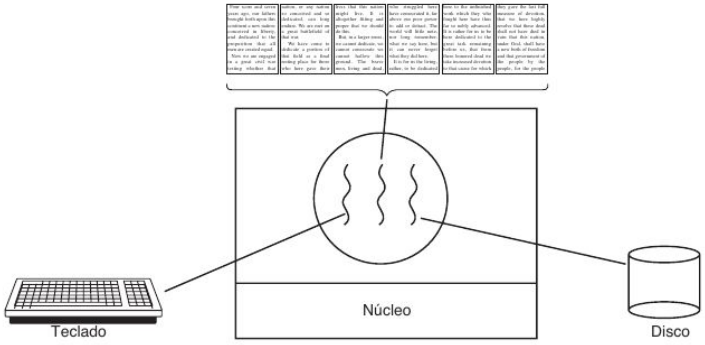
- Em diversas aplicações há múltiplas atividades ao mesmo tempo
- Essas múltiplas atividades podem ser decompostas em atividades paralelas
- ***CPU-bound* e *I/O-bound* podem se sobrepor**
  - Modo de aceleração de aplicações

Threads - Vantagens

- São **mais rápidas de criar** e destruir que processos
  - Até 100 vezes mais rápidas
- Úteis em sistemas com múltiplas CPUs
  - **Possibilidade de paralelismo real**

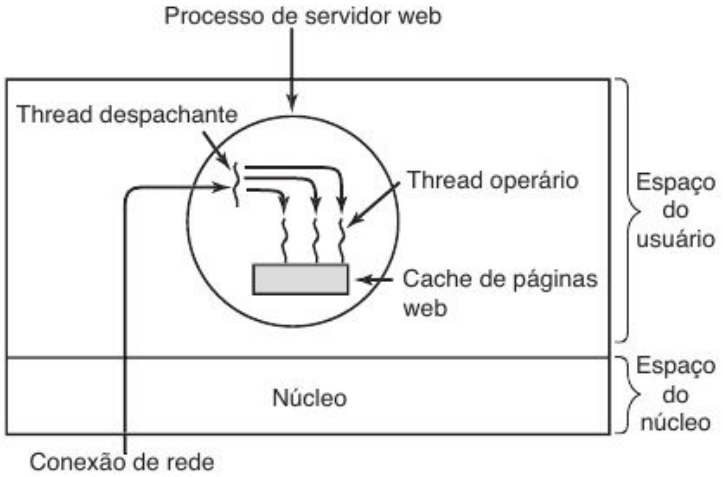
Threads - Exemplo

- **Processador de Texto**
  - Todas as linhas de execução precisam acessar o documento
    - Processos separados não funcionam
  - Diferentes execuções para interagir com o usuário, reformatar documento, *backup* automático, etc

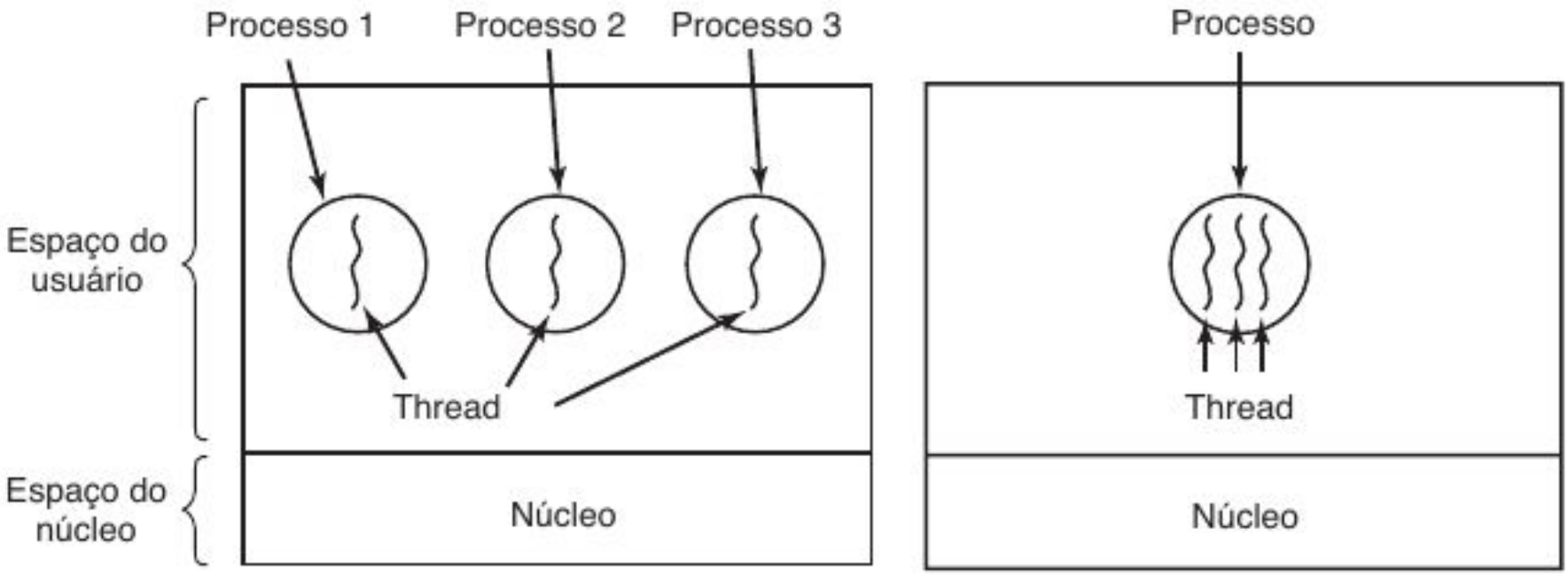


Threads - Exemplo

- **Servidor Web**
  - Ler requisições de trabalho (*thread* despachante)
  - Operários aguardando trabalho (*thread* operário)
    - Quando acordado verifica *cache* ou disco

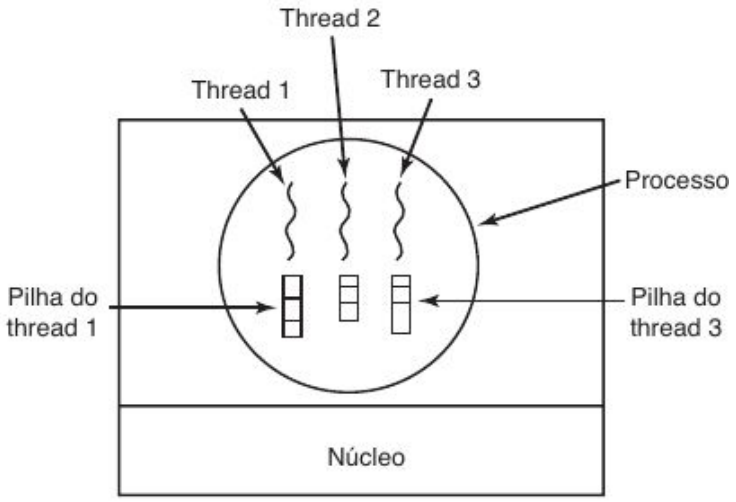


Processos vs *Threads*



Processos vs *Threads*

- Cada ***thread*** **tem sua própria pilha** de execução (para poderem chamar rotinas diferentes)
- **Compartilham o espaço de endereçamento** e todos os seus dados



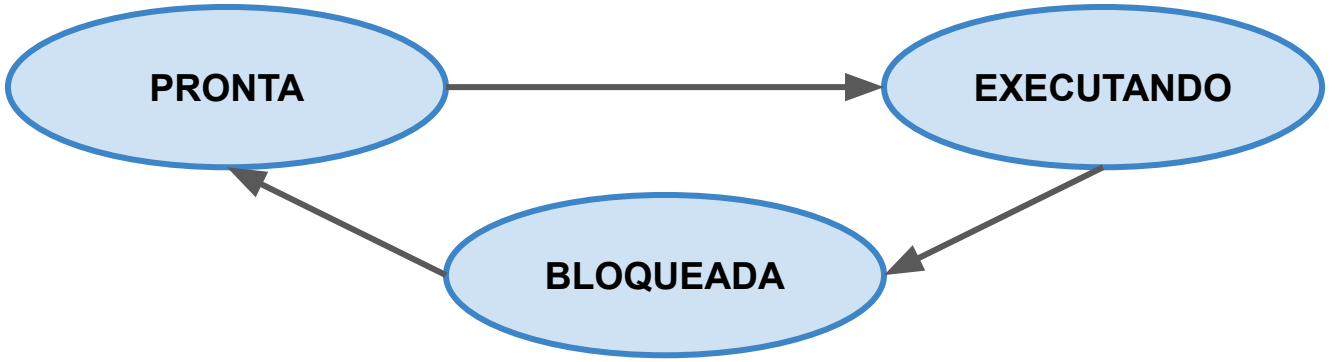
Itens por processo	Itens por thread
Espaço de endereçamento	Contador de programa
Variáveis globais	Registradores
Arquivos abertos	Pilha
Processos filhos	Estado
Alarmes pendentes	
Sinais e tratadores de sinais	
Informação de contabilidade	

Problemas com *Threads*

- Por *threads* terem acesso a qualquer endereço de memória dentro do espaço de endereçamentos do processo:
  - Uma *thread* pode **ler e escrever em variáveis globais ou na pilha ou na pilha de outra *thread***
    - Exemplo:
      - `a = b + c;`
      - `x = a + y;`
- Necessidade de **sincronizar** a execução

Estados de *Threads*

- A entidade que realmente executa é a *thread*
  - Processo é a apenas o ambiente
- *Threads* possuem os seguintes estados:



Níveis de Implementação de *Threads*

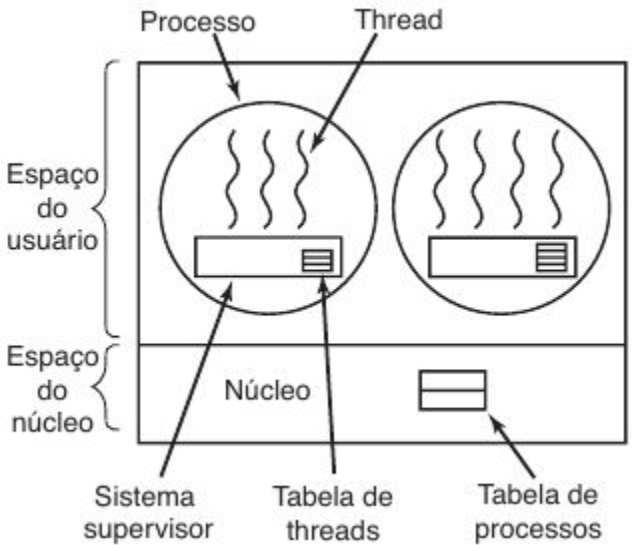
- Threads podem ser implementadas de três formas principais:
  - **No Modo Usuário**
  - **No Modo Núcleo**
  - **De maneira Híbrida**

Threads no Modo Usuário

- Implementadas **totalmente no espaço de usuário**
- Feito por meio de uma biblioteca
  - Criação, exclusão, execução, etc
- **Criação e escalonamento** são realizados **sem o conhecimento do Kernel**
- Para o Kernel, só existe um processo com uma única linha de execução (*monothread*)

Threads no Modo Usuário

- O kernel escolhe um processo e passa o controle a ele
- O processo escolhe a linha de execução (thread) que irá executar
- A gerência da thread fica no espaço de usuário
- O núcleo só realiza escalonamento no nível de processo



Threads no Modo Núcleo

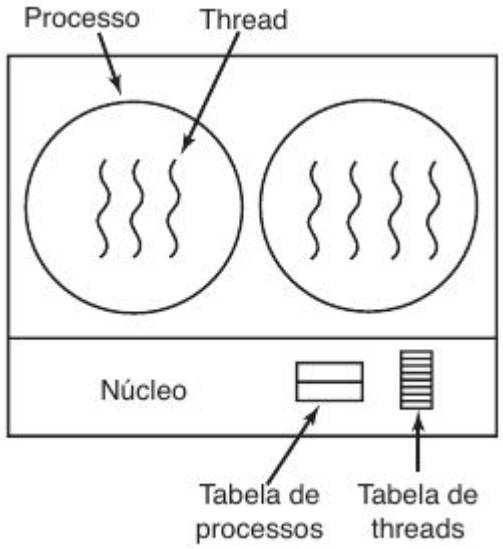
- Suportada **diretamente pelo Sistema Operacional**
- **Criação e escalonamento são feitos pelo kernel**
- O **kernel possui**, além da tabela de processos, uma **tabela de *threads***, com todas as threads do sistema
- Nesse modelo, os escalonamentos mais utilizados são *Round-Robin* e Prioridade

Threads no Modo Núcleo

- Consequências do modelo:
  - **Gerenciar threads fica mais caro**, pois agora envolve alternância entre modo usuário e modo núcleo
  - Mudança de contexto pode envolver *threads*
  - Criar e destruir threads no núcleo é mais caro

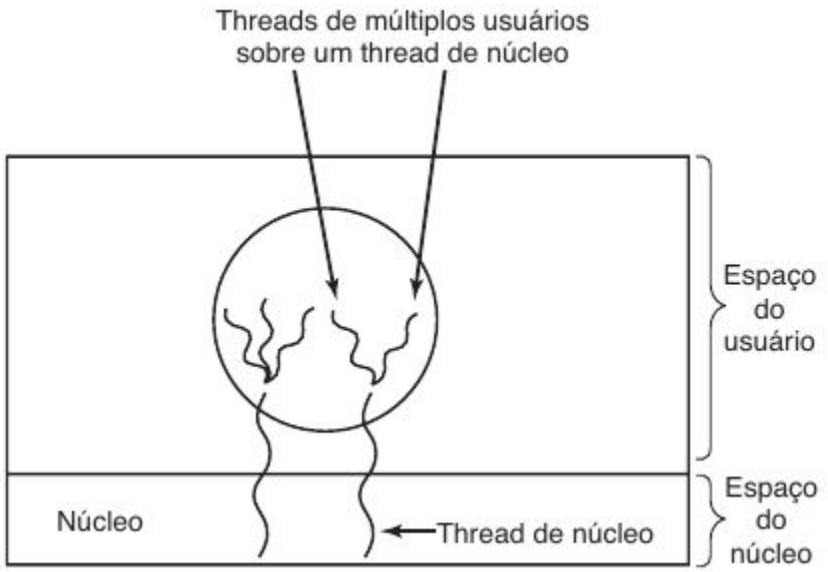
Threads no Modo Núcleo

- O núcleo escolhe a *thread* diretamente
- A *thread* é quem recebe o *quantum*
- *Thread* bloqueada por E/S não bloqueia o processo que a envolve
- Permite múltiplas *threads* em paralelo



Threads Híbridas

- Seguem o modelo N para M
- N *threads* de usuário são mapeadas em M threads de núcleo
- Programador decide quantas *threads* de núcleo usar e quantas *threads* de usuário multiplexar
- Kernel está consciente apenas das *threads* de núcleo

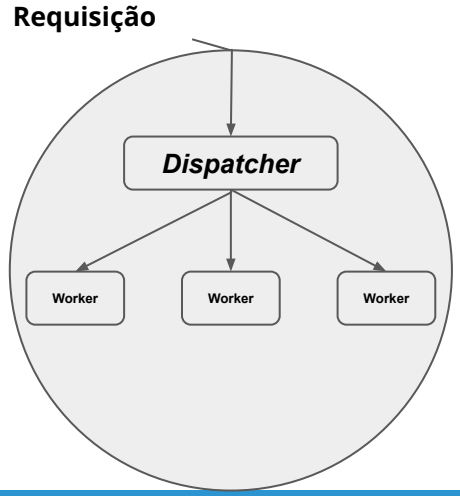


Modelos de Execução de *Threads*

- Define como *threads* irão se organizar para resolver problemas
- Dependente do problema a ser resolvido
- Em servidores, por exemplo, o modelo pode ser:
  - **De threads dinâmicas**:
    - Uma thread é criada para tratar cada requisição
  - **De threads estáticas**:
    - Número de threads é fixo

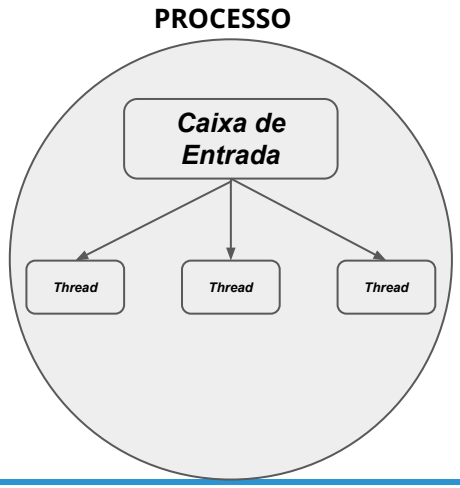
Modelos de Execução de *Threads* - *Dispatcher/Worker*

- Modelo do tipo estático
- Thread *dispatcher* recebe todas as requisições por trabalho
- Aciona uma *thread worker* e entrega o trabalho a ser feito



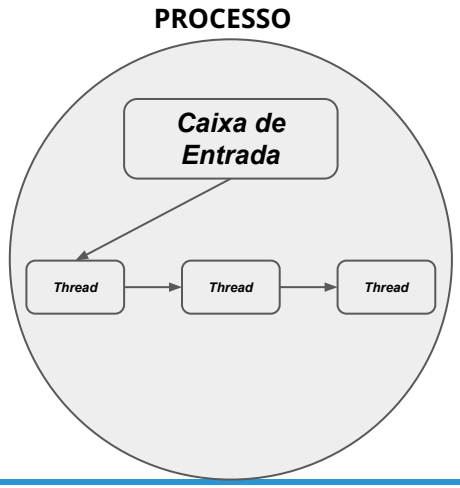
Modelos de Execução de *Threads* - Time *(Team)*

- Modelo do tipo estático
- *Threads* são autônomas
- Consultam um buffer, obtém uma requisição e a executa



Modelos de Execução de *Threads* - *Pipeline*

- Modelo estático
- *Threads* possuem tarefas específicas
- Tarefa vem da thread anterior



Bibliotecas de *Threads*

- Bibliotecas de *threads* fornecem ao programador uma **API para gerenciamento de *threads***
- As duas bibliotecas mais comuns são:
  - ***POSIX Threads***
  - **Java**
    - Concebida já com *threads*
    - Máquina Virtual: *thread main*, coletor de lixo, finalização de objetos, etc...
    - Cada *thread* é associada a uma instância da classe *Thread*

Padrão IEEE POSIX

- Portable Operating System Interface (POSIX) é uma família de normas definidas pelo IEEE para a manutenção de compatibilidade entre sistemas operacionais
- Busca garantir a portabilidade do código-fonte entre sistemas que atendam a norma internacional IEEE POSIX 1003.1 C
- *Threads* pela norma POSIX podem ser implementadas em nível de usuário ou de kernel

Padrão IEEE POSIX - Algumas Implementações

- LinuxThread (1996): nível de kernel e não totalmente compatível com o padrão POSIX
- GNU Pth (1999): nível de usuário e compatível com o padrão POSIX
- NGPT (2002): Misto, baseado no GNU Pth e compatível (descontinuada)
- NPTL (2002): nível de kernel, POSIX e o padrão atual do Linux
- PM2/Marcel (2011): Misto, compatível e com extensões para alto desempenho

Conclusão